



National Technical University of Athens

School of Electrical and Computer Engineering

8th Semester

VLSI Digital Systems

Laboratory Exercise 2

Dimitris Vatalas

(Student ID: 03121093)

Aristotelis Georgios Dritsas

(Student ID: 03122127)

Athens, March 2026

Contents

Problem 1: Synchronous Full Adder (FA) with Behavioral Description	2
Problem 2: 4-bit Synchronous Carry Propagation Adder Using Pipelining	6
Problem 3: 4-bit Systolic Carry Propagation Multiplier	15
Bonus: Programming the Zybo Board and Experimentation	25

Problem 1: Synchronous Full Adder (FA) with Behavioral Description

The VHDL code describing the synchronous Full Adder and its behavioral architecture is presented below:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Sync_FA is
    Port (
        --FA
        a, b, cin: in std_logic;
        cout, sum: out std_logic;

        --FF
        clk: in std_logic
    );
end Sync_FA;

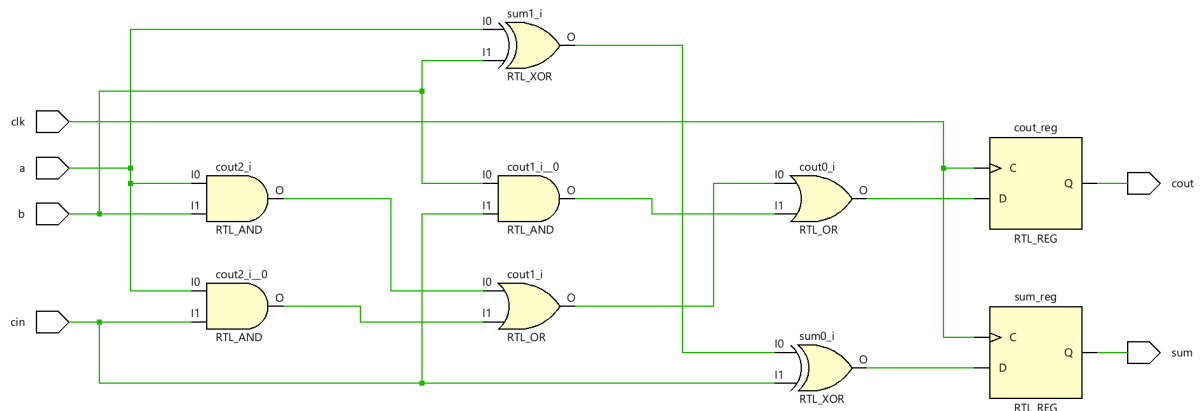
architecture Behavioral of Sync_FA is

begin

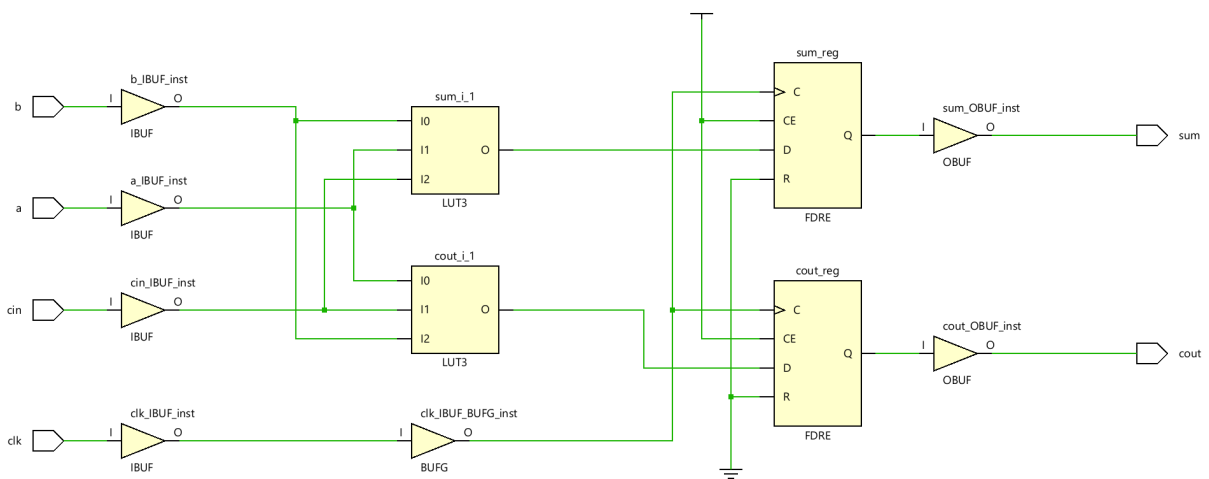
    process(clk)
    begin
        if(rising_edge(clk)) then
            sum <= a xor b xor cin;
            cout <= (a and b) or (a and cin) or (b and cin);
        end if;
    end process;

end Behavioral;
```

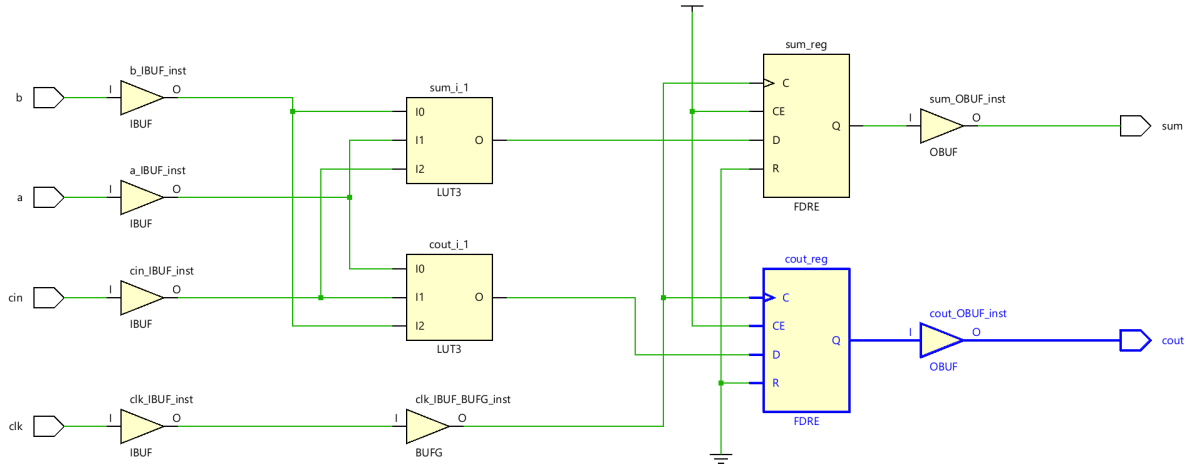
Using the Elaborate Design option, we obtain the following schematic:



Using synthesis, we obtain the following schematic:



By using the timing summary of our circuit, we can determine its critical path, which is shown below:



As shown below, the propagation delay of the circuit's critical path is 4.076 ns.

Unconstrained Paths - NONE - NONE - Setup														
Name	Slack	Levels	Routes	High Fanout	From	To	Total ... 1	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Clock Uncertainty
Path 1	∞	2	2	1	cout_reg/C	cout	4.076	3.276	0.800	∞				0.000
Path 2	∞	2	2	1	sum_reg/C	sum	4.076	3.276	0.800	∞				0.000
Path 3	∞	2	3	2	a	sum_reg/D	1.932	1.132	0.800	∞	input port clock			0.000
Path 4	∞	2	3	2	b	cout_reg/D	1.906	1.106	0.800	∞	input port clock			0.000

Identifying the critical path is important because it allows us to determine which data path is the most time-consuming in our design. Therefore, if timing improvements are required, we know which path needs to be optimized. In addition, the maximum propagation delay directly affects the minimum clock period, and consequently the maximum operating frequency of the circuit.

To simulate and verify the correct operation of our circuit, we use the following testbench:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Sync_FA_tb is
    -- Port ( );
end Sync_FA_tb;

architecture tb of Sync_FA_tb is

    component Sync_FA is
        Port (
            a, b, cin, clk : in std_logic;
            cout, sum : out std_logic
        );
    end component;

    signal a_tb, b_tb, cin_tb, clk_tb, cout_tb, sum_tb : std_logic := '0';

    constant CLOCK_PERIOD : time := 10 ns;

begin

```

```

DUT : Sync_FA
  port map
  (
    a => a_tb,
    b => b_tb,
    cin => cin_tb,
    cout => cout_tb,
    sum => sum_tb,
    clk => clk_tb
  );

-- Clock generator
CLOCK_GEN : process
begin
  clk_tb <= '0';
  wait for CLOCK_PERIOD / 2;
  clk_tb <= '1';
  wait for CLOCK_PERIOD / 2;
end process;

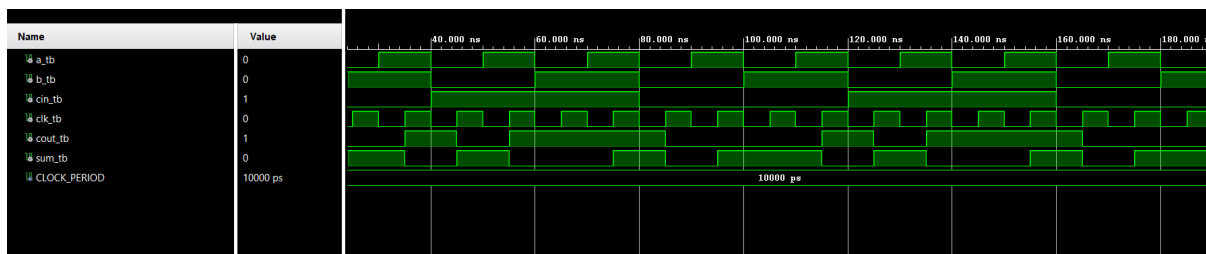
-- Input A
IN_A : process
begin
  a_tb <= '0';
  wait for CLOCK_PERIOD;
  a_tb <= '1';
  wait for CLOCK_PERIOD;
end process;

-- Input B
IN_B : process
begin
  b_tb <= '0';
  wait for CLOCK_PERIOD * 2;
  b_tb <= '1';
  wait for CLOCK_PERIOD * 2;
end process;

-- Carry input
IN_CARRY : process
begin
  cin_tb <= '0';
  wait for CLOCK_PERIOD * 4;
  cin_tb <= '1';
  wait for CLOCK_PERIOD * 4;
end process;

end tb;

```

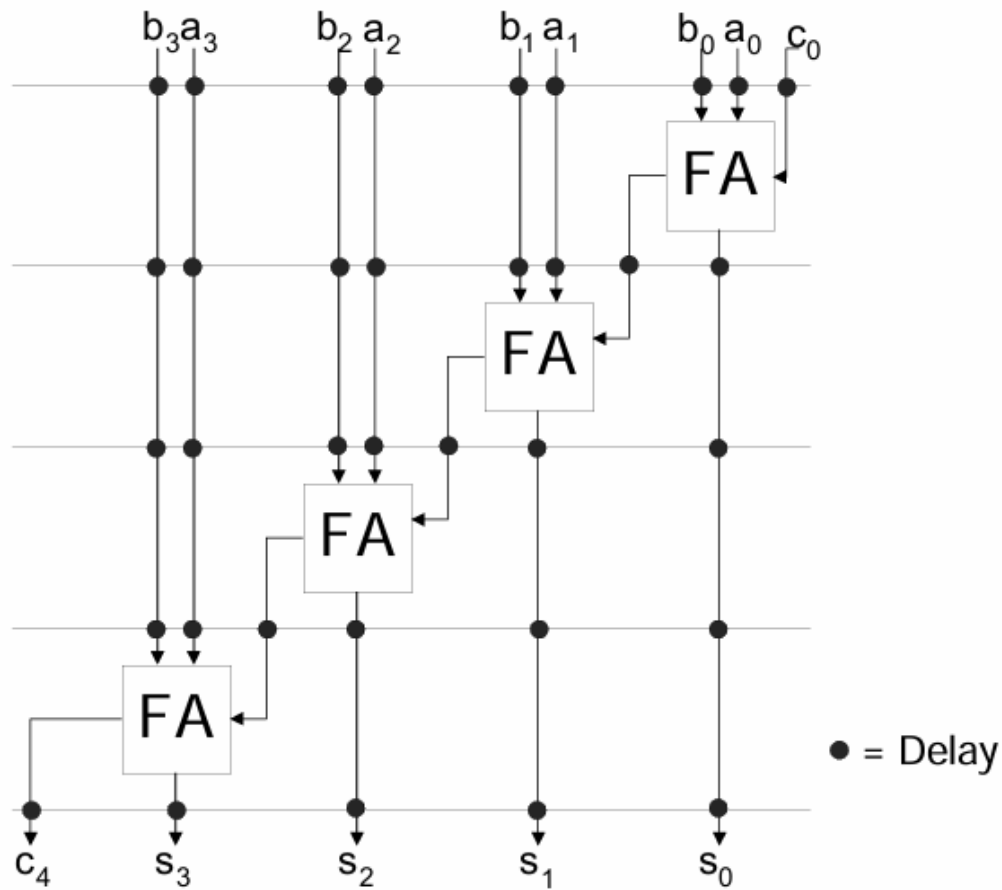


We observe that the circuit produces the correct results for all possible input combinations.

Problem 2: 4-bit Synchronous Carry Propagation Adder Using Pipelining

The design of the 4-bit Synchronous Carry Propagation Adder, using Full Adders and pipelining techniques, was based on the schematic shown below. However, a small modification was made: no additional delay was introduced at the inputs a_0 and b_0 , as this was considered unnecessary. This choice requires corresponding adjustments in the subsequent pipeline stages, which are not analyzed further, since they effectively correspond to removing one clock cycle of delay from each input pair. This modification does not affect the correct operation of the circuit, since bits a_0 and b_0 are used directly in the first pipeline stage and therefore do not need to be temporally aligned with data from previous stages.

In general, and in order to explicitly address the question in the assignment concerning the delays that were introduced, additional registers (delays) were inserted at the inputs, between the successive stages of the circuit, and at the outputs, as shown in the following diagram, in order to implement the pipelined operation. These registers ensure the temporal alignment of the data, so that each stage receives the appropriate signals at the correct time. In this way, correct carry propagation and proper operation of the adder are maintained.



For the implementation of the adder, a module implementing a shift register with a variable delay was used, as shown below:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity shift_reg is
    generic (
        DELAY : integer := 1 -- number of clock cycles to delay
    );
    Port (
        clk : in  std_logic;
        d   : in  std_logic;
        q   : out std_logic
    );
end shift_reg;

architecture Behavioral of shift_reg is
    signal tmp : std_logic_vector(DELAY-1 downto 0);
begin
    process(clk)
    begin
        if rising_edge(clk) then
            tmp(0) <= d;
        end if;
    end process;
end Behavioral;

```

```

        for i in 1 to DELAY-1 loop
            tmp(i) <= tmp(i-1);
        end loop;
    end if;
end process;

q <= tmp(DELAY-1);
end Behavioral;

```

The VHDL code describing the adder circuit and its architecture is presented below:

```

library IEEE;library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Sync_4FA is
    Port (
        a, b: in  std_logic_vector(3 downto 0);
        cin: in  std_logic;
        sum: out std_logic_vector(3 downto 0);
        cout: out std_logic;
        clk: in  std_logic
    );
end Sync_4FA;

architecture Behavioral of Sync_4FA is

    component Sync_FA is
        Port (
            a, b, cin: in std_logic;
            sum, cout: out std_logic;
            clk: in std_logic
        );
    end component;

    component shift_reg is
        generic (DELAY : integer := 1);
        Port (
            clk : in  std_logic;
            d   : in  std_logic;
            q   : out std_logic
        );
    end component;

    signal c : std_logic_vector(4 downto 0);

    signal a_pipe, b_pipe : std_logic_vector(3 downto 0);
    signal sum_pipe : std_logic_vector(3 downto 0);

begin

    c(0) <= cin;

    -- bit 0 (LSB), no delay needed
    a_pipe(0) <= a(0);
    b_pipe(0) <= b(0);

```

```

-- bits 1..3 use shift registers
gen_shift_in: for i in 1 to 3 generate

    SR_a: shift_reg
        generic map (DELAY => i)
        port map (
            clk => clk,
            d   => a(i),
            q   => a_pipe(i)
        );

    SR_b: shift_reg
        generic map (DELAY => i)
        port map (
            clk => clk,
            d   => b(i),
            q   => b_pipe(i)
        );

end generate;

gen_FA: for i in 0 to 3 generate
begin
    FA_i: Sync_FA
        port map(
            a => a_pipe(i),
            b => b_pipe(i),
            cin => c(i),
            sum => sum_pipe(i),
            cout => c(i+1),
            clk => clk
        );
end generate;

-- bit 4 (MSB), no delay needed
sum(3) <= sum_pipe(3);

-- bits 1..3 use shift registers
gen_shift_out: for i in 0 to 2 generate

    SR_sum: shift_reg
        generic map (DELAY => 3 - i)
        port map (
            clk => clk,
            d   => sum_pipe(i),
            q   => sum(i)
        );

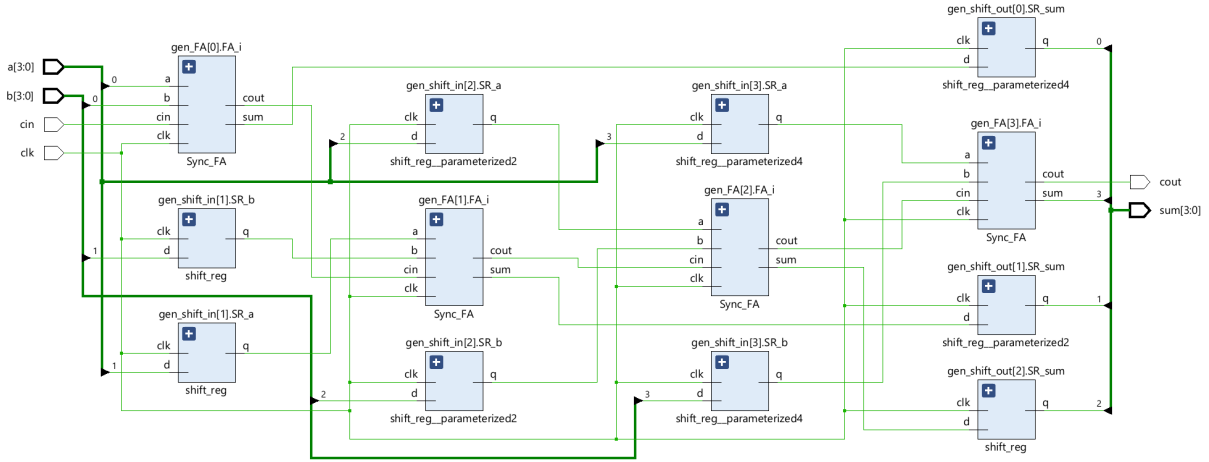
end generate;

cout <= c(4);

end Behavioral;

```

Using the Elaborate Design option, we obtain the following schematic:



In the schematic above, we can observe the four Full Adders, as well as the additional buffers used to introduce the required delays.

To simulate and verify the correct operation of our circuit, we use the following testbench:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.std_logic_unsigned.all;

entity Sync_4FA_TB is
-- Port ( );
end Sync_4FA_TB;

architecture tb of Sync_4FA_TB is

component Sync_4FA is
  Port (
    --FA
    a, b: in  std_logic_vector(3 downto 0);
    cin: in  std_logic;
    sum: out std_logic_vector(3 downto 0);
    cout: out std_logic;

    --FF
    clk: in  std_logic
  );
end component;

signal a, b, sum: std_logic_vector(3 downto 0) := "0000";
signal cin, clk, cout: std_logic := '0';

constant CLOCK_PERIOD : time := 10 ns;

begin

  DUT : Sync_4FA
    port map
    (
      a => a,
      b => b,
      cin => cin,
      cout => cout,
      sum => sum,
      clk => clk
    );

  -- Clock generator
  CLOCK_GEN : process
  begin
    clk <= '0';
    wait for CLOCK_PERIOD / 2;
    clk <= '1';
    wait for CLOCK_PERIOD / 2;
  end process;

  -- Input A
  IN_A : process
  begin
    a <= a + 1;
```

```

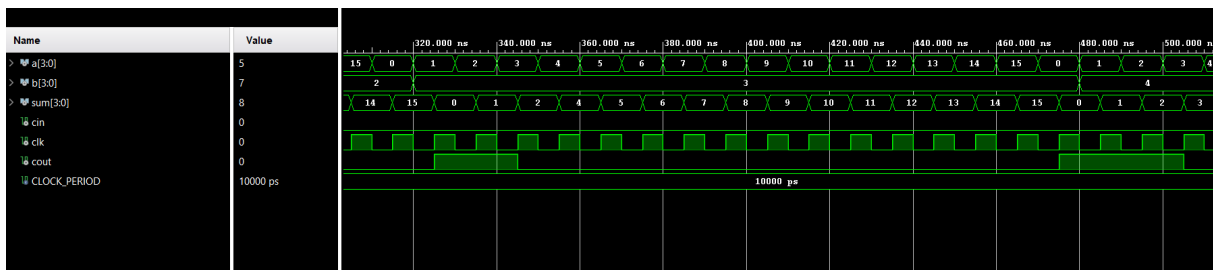
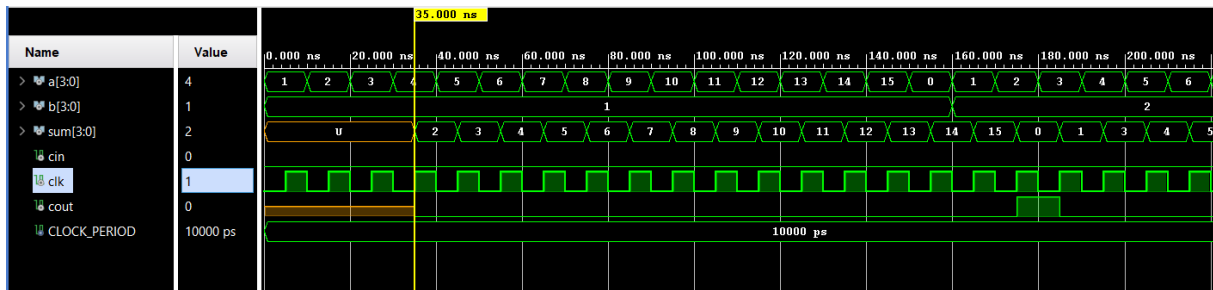
        wait for CLOCK_PERIOD;
end process;

-- Input B
IN_B : process
begin
    b <= b + 1;
    wait for CLOCK_PERIOD * 16;
end process;

-- Carry input
IN_CARRY : process
begin
    cin <= '0';
    wait for CLOCK_PERIOD * 16 * 16;
    cin <= '1';
    wait for CLOCK_PERIOD * 16 * 16;
end process;

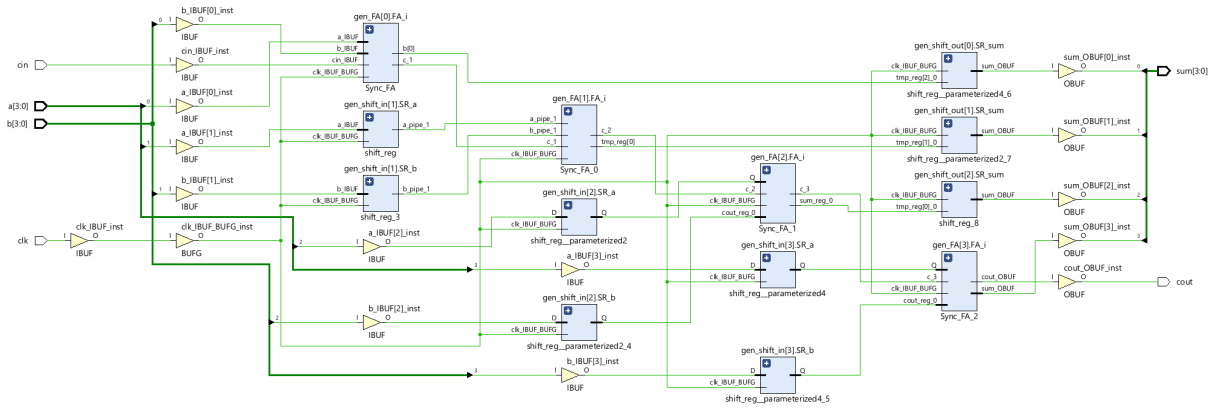
end tb;

```

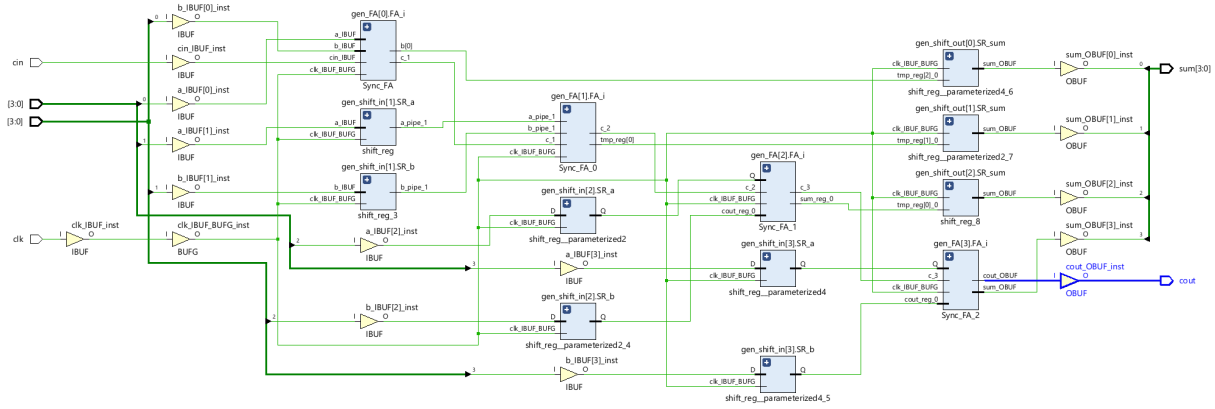


In the first image, we observe that the first valid result appears at the fourth rising edge of the clock, corresponding to a latency of four clock cycles, as expected due to the pipeline introduced for synchronization purposes. From the fourth cycle onward, the circuit produces one new correct result at every clock cycle, as confirmed by both of the presented images. Therefore, the correct operation of the circuit is verified.

Using synthesis, we obtain the following schematic:



Using the timing summary of our circuit, we can determine its critical path, which is shown below:



As shown below, the propagation delay of the circuit's critical path is 4.076 ns. It is worth noting that, due to the use of pipelining, this delay corresponds to a single pipeline stage rather than a path extending from the circuit input to its output. In addition, although this is a synchronous circuit, the slack appears as infinite. We assume that this is due to the fact that appropriate timing constraints have not been defined for the design, and therefore the tool does not know the timing requirements that need to be satisfied.

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Clock Uncertainty
Path 1	∞	2	2	1	gen_FA[3].FA_i/cout_reg/C	cout	4.076	3.276	0.800	∞				0.000
Path 2	∞	2	2	1	gen_shift_out[2].um/tmp_reg[0]/C	sum[2]	4.076	3.276	0.800	∞				0.000
Path 3	∞	2	2	1	gen_FA[3].FA_i/sum_reg/C	sum[3]	4.076	3.276	0.800	∞				0.000
Path 4	∞	2	2	1	gen_shift_out[0].um/tmp_reg[2]/C	sum[0]	4.058	3.258	0.800	∞				0.000
Path 5	∞	2	2	1	gen_shift_out[1].um/tmp_reg[1]/C	sum[1]	4.058	3.258	0.800	∞				0.000
Path 6	∞	2	3	2	a[0]	gen_shift_out[0].p_reg[1].sr[3]/D	2.265	1.132	1.133	∞	input port clock			0.000
Path 7	∞	2	3	2	b[0]	gen_FA[0].FA_i/cout_reg/D	1.906	1.106	0.800	∞	input port clock			0.000
Path 8	∞	2	2	2	gen_shift_in[1].SR_b/tmp_reg[0]/C	gen_shift_out[1].p_reg[0].sr[2]/D	1.862	0.777	1.085	∞				0.000
Path 9	∞	1	2	1	a[1]	gen_shift_in[1].SR_a/tmp_reg[0]/D	1.782	0.982	0.800	∞	input port clock			0.000
Path 10	∞	1	2	1	b[1]	gen_shift_in[1].SR_b/tmp_reg[0]/D	1.782	0.982	0.800	∞	input port clock			0.000

From the analysis of the previous laboratory exercise, it was determined that for the simple 4-bit Full Adder, the total critical path delay is 5.970 ns (from input to output), whereas for the synchronous (pipelined) 4-bit adder, the critical path delay was found to be 4.076 ns.

Since the clock period is determined by the critical path, the maximum operating frequencies are given by:

$$f_{\max, \text{FA}} = \frac{1}{5.970 \text{ ns}} \approx 167.5 \text{ MHz}$$
$$f_{\max, \text{pipeline}} = \frac{1}{4.076 \text{ ns}} \approx 245.3 \text{ MHz}$$

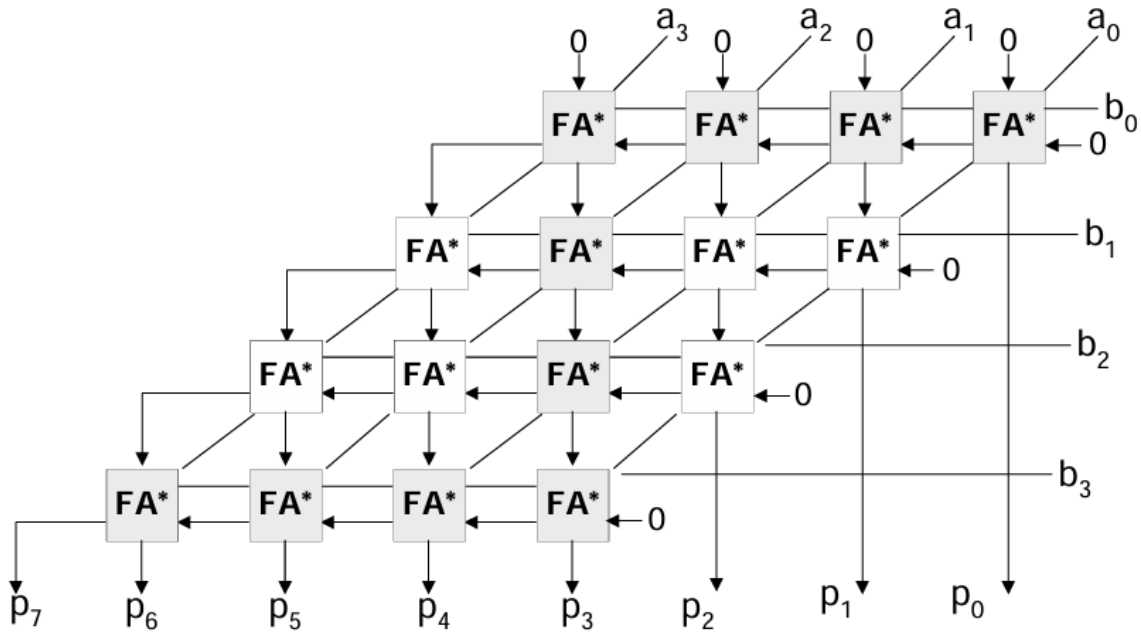
We observe that the pipelined implementation allows for a higher operating frequency, since the critical path delay has been reduced. On the other hand, the use of pipelining introduces latency, since several clock cycles are required before the first result appears (four cycles in our case).

Therefore, if the pipeline is not fully utilized, i.e., if we are interested in producing only a single result, the simple implementation from the previous laboratory exercise is more efficient, since it completes the computation within a single cycle. In contrast, under continuous data-flow conditions, the pipelined implementation is superior, since after the initial latency it can produce one result per clock cycle, achieving higher throughput.

Regarding resource utilization, the pipelined implementation requires more hardware resources, mainly additional registers used to implement the pipeline stages. This increases the complexity and hardware cost of the circuit. Thus, an expected trade-off between throughput and hardware cost can be observed.

Problem 3: 4-bit Systolic Carry Propagation Multiplier

The design of the 4-bit systolic carry propagation multiplier is based on the circuit shown below:



It should be noted that, in order for a circuit to be characterized as systolic, the following basic requirements must be satisfied:

- The cells composing the circuit are identical and implement the same basic function.
- Continuous data flow between the cells is supported.
- Communication between the cells is synchronized, with storage elements (registers or latches) inserted to ensure the gradual propagation of data through the pipeline.

Initially, the basic structural unit of the multiplier must be designed. This unit consists of a Full Adder combined with an AND gate, as shown in the figure below. More specifically, this basic cell is responsible for calculating the partial product of two bits using the AND gate. The resulting partial product is then added to the corresponding partial sum and the carry from the previous stage using the Full Adder. In addition, as shown below, the cell contains flip-flops that introduce the necessary timing delays to ensure proper synchronization of the data between successive stages and correct pipeline operation. In this way, the results are forwarded to the next stage with a controlled delay, maintaining the temporal alignment of the system.


```

        q    : out std_logic
    );
end component;

signal ab : std_logic;

begin

    ab <= a and b;

    process(clk)
    begin
        if(rising_edge(clk)) then
            sum <= ab xor sin xor cin;
            cout <= (ab and sin) or (ab and cin) or (sin and cin);
        end if;
    end process;

    SR_a: shift_reg
        generic map (DELAY => 2)
        port map (
            clk => clk,
            d   => a,
            q   => a_delayed
        );

    SR_b: shift_reg
        generic map (DELAY => 1)
        port map (
            clk => clk,
            d   => b,
            q   => b_delayed
        );

end Behavioral;

```

The VHDL code describing the multiplier and its structural architecture is presented below:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Syst_4Mult is
    Port (
        a, b : in  std_logic_vector(3 downto 0);
        p    : out std_logic_vector(7 downto 0);
        clk  : in  std_logic
    );
end Syst_4Mult;

architecture Behavioral of Syst_4Mult is

    component Sync_FA_MULT is
        Port (
            -- FA
            a, b, cin, sin : in  std_logic;

```

```

        cout, sum      : out std_logic;
        a_delayed, b_delayed : out std_logic;

        -- FF
        clk : in  std_logic
    );
end component;

component shift_reg is
    generic (DELAY : integer := 1);
    Port (
        clk : in  std_logic;
        d   : in  std_logic;
        q   : out std_logic
    );
end component;

type matrix_5x5 is array (0 to 4) of std_logic_vector(4 downto 0);
signal sout_wire, cout_wire : matrix_5x5;
signal a_wire, b_wire      : matrix_5x5;
signal cout_del            : std_logic_vector(2 downto 0);
signal p_zero_to_two      : std_logic_vector(3 downto 0);

begin

    a_wire(0)(0) <= a(0);
    b_wire(0)(0) <= b(0);

    gen_rows: for i in 0 to 3 generate
        gen_cols: for j in 0 to 3 generate

            begin

                first_cell: if (i = 0 and j = 0) generate
                    Sync_FA_MULT_inst: Sync_FA_MULT
                        port map (
                            a  => a_wire(i)(j),
                            b  => b_wire(i)(j),
                            clk => clk,

                            cin => '0',
                            sin => '0',
                            sum => p_zero_to_two(i),

                            cout      => cout_wire(i)(j+1),
                            a_delayed => a_wire(i+1)(j),
                            b_delayed => b_wire(i)(j+1)
                        );
                end generate;

                top_row: if (i = 0 and j > 0) generate
                    Sync_FA_MULT_inst: Sync_FA_MULT
                        port map (
                            a  => a_wire(i)(j),
                            b  => b_wire(i)(j),

```

```

        cin => cout_wire(i)(j),
        sin => '0',
        clk => clk,

        cout      => cout_wire(i)(j+1),
        sum       => sout_wire(i+1)(j-1),
        a_delayed => a_wire(i+1)(j),
        b_delayed => b_wire(i)(j+1)
    );
end generate;

left_column: if (i > 0 and j = 3) generate
    Sync_FA_MULT_inst: Sync_FA_MULT
        port map (
            a  => a_wire(i)(j),
            b  => b_wire(i)(j),
            clk => clk,

            cin => cout_wire(i)(j),
            sin => cout_del(i-1),
            sum => sout_wire(i+1)(j-1),

            cout      => cout_wire(i)(j+1),
            a_delayed => a_wire(i+1)(j),
            b_delayed => b_wire(i)(j+1)
        );
end generate;

inner_cells: if (i > 0 and (j > 0 and j < 3)) generate
    Sync_FA_MULT_inst: Sync_FA_MULT
        port map (
            a  => a_wire(i)(j),
            b  => b_wire(i)(j),
            cin => cout_wire(i)(j),
            sin => sout_wire(i)(j),
            sum => sout_wire(i+1)(j-1),

            cout      => cout_wire(i)(j+1),
            a_delayed => a_wire(i+1)(j),
            b_delayed => b_wire(i)(j+1),
            clk => clk
        );
end generate;

right_column: if (i > 0 and j = 0) generate
    Sync_FA_MULT_inst: Sync_FA_MULT
        port map (
            a  => a_wire(i)(j),
            b  => b_wire(i)(j),
            clk => clk,

            cin => '0',
            sin => sout_wire(i)(j),
            sum => p_zero_to_two(i),

```

```

        cout      => cout_wire(i)(j+1),
        a_delayed => a_wire(i+1)(j),
        b_delayed => b_wire(i)(j+1)
    );
end generate;

end generate;
end generate;

gen_shift_in_a: for i in 0 to 2 generate
    SR_a: shift_reg
        generic map (DELAY => i+1)
        port map (
            clk => clk,
            d   => a(i+1),
            q   => a_wire(0)(i+1)
        );
end generate;

gen_shift_out_3_to_7: for i in 0 to 1 generate
    SR_p_1: shift_reg
        generic map (DELAY => 2-i)
        port map (
            clk => clk,
            d   => sout_wire(4)(i),
            q   => p(4+i)
        );
end generate;

gen_shift_out_0_to_3: for i in 0 to 3 generate
    SR_p_2: shift_reg
        generic map (DELAY => 3 + 2*i)
        port map (
            clk => clk,
            d   => p_zero_to_two(3-i),
            q   => p(3-i)
        );
end generate;

gen_shift_cout: for i in 0 to 2 generate
    SR_cout: shift_reg
        generic map (DELAY => 1)
        port map (
            clk => clk,
            d   => cout_wire(i)(4),
            q   => cout_del(i)
        );
end generate;

gen_shift_in_b: for i in 0 to 2 generate
    SR_bt: shift_reg
        generic map (DELAY => 2*(i+1))
        port map (
            clk => clk,
            d   => b(i+1),

```

```

        q    => b_wire(i+1)(0)
    );
end generate;

P(6) <= sout_wire(4)(2);
p(7) <= cout_wire(3)(4);

end Behavioral;

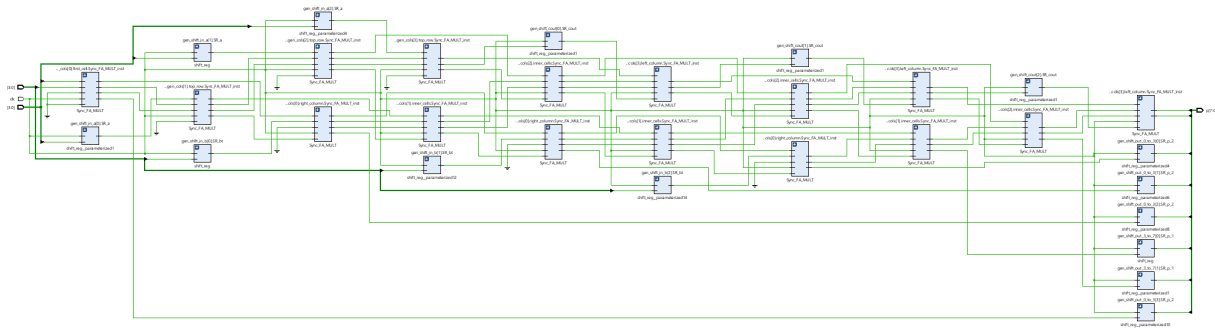
```

The multiplier consists of 16 Full Adders* arranged across 10 levels. In addition, the necessary registers for the delays required at the input and output of the multiplier have been designed. As mentioned above, the intermediate delays between the Full Adders* are implemented within the basic Full Adder* cells themselves.

It should be noted that four of the 16 Full Adders* could be omitted, since in the first row each Full Adder* essentially performs only the AND-gate operation, as one of the two inputs to the Full Adder* is constantly 0. With this optimization, the pipeline latency could also be reduced, saving additional clock cycles.

However, the use of uniform structural units throughout the circuit was intentionally selected in order to preserve the regularity of the architecture, which is a fundamental design principle of systolic architectures. Furthermore, minimizing the number of pipeline cycles was not a primary objective of this implementation, so the more uniform design described above was preferred.

Using the Elaborate Design option, we obtain:



To verify the correct operation of the multiplier, the following testbench is used. Initially, several representative multiplication cases are executed. Subsequently, the testbench is extended to perform an exhaustive verification by testing all possible input combinations. It should be noted that, in practice, the simulation was not run until all cases had been completed. Nevertheless, theoretically, given sufficient simulation time, the testbench would cover the entire set of possible input combinations.

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity fa_mul_tb is
end fa_mul_tb;

architecture tb of fa_mul_tb is

    component Syst_4Mult
        Port (
            a, b : in  std_logic_vector(3 downto 0);
            p    : out std_logic_vector(7 downto 0);
            clk  : in  std_logic
        );
    end component;

    signal a    : std_logic_vector(3 downto 0) := (others => '0');
    signal b    : std_logic_vector(3 downto 0) := (others => '0');
    signal p    : std_logic_vector(7 downto 0);
    signal clk  : std_logic := '0';

    constant CLOCK_PERIOD : time := 10 ns;

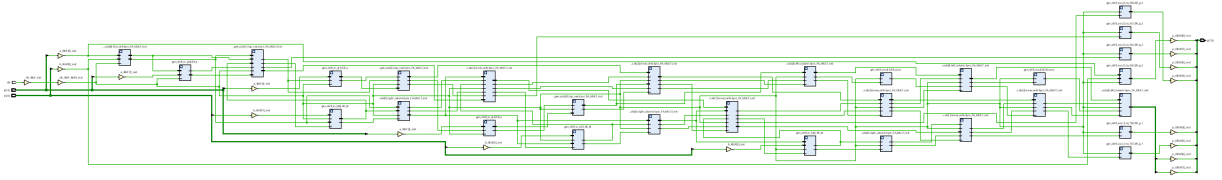
begin

    UUT: Syst_4Mult
        port map (
            a => a,
            b => b,
            p => p,
            clk => clk
        );

    CLK_GEN : process
    begin
        clk <= '0';
        wait for CLOCK_PERIOD / 2;
        clk <= '1';
        wait for CLOCK_PERIOD / 2;
    end process;

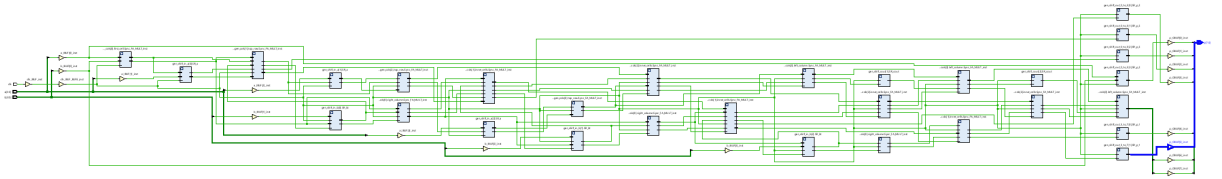
    STIMULUS : process
    begin
        -- Test case 1: 3 * 2 = 6
        a <= "0011";
        b <= "0010";
        wait for CLOCK_PERIOD;
    end process;
end tb;

```

To determine the critical path of the complete circuit and its corresponding propagation delay using the timing summary, we obtain:

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Clock Uncertainty
Path 1	∞	2	2	1	gen_shift_out_3_1/tmp_reg[0]/C	p[5]	4.076	3.276	0.800	∞				0.000
Path 2	∞	2	2	1	gen_rows[3].gen_inst/sum_reg/C	p[6]	4.076	3.276	0.800	∞				0.000
Path 3	∞	2	2	1	gen_rows[3].gen_inst/cout_reg/C	p[7]	4.076	3.276	0.800	∞				0.000
Path 4	∞	2	2	1	gen_shift_out_0_2/tmp_reg[8]/C	p[0]	4.058	3.258	0.800	∞				0.000
Path 5	∞	2	2	1	gen_shift_out_0_2/tmp_reg[6]/C	p[1]	4.058	3.258	0.800	∞				0.000
Path 6	∞	2	2	1	gen_shift_out_0_2/tmp_reg[4]/C	p[2]	4.058	3.258	0.800	∞				0.000
Path 7	∞	2	2	1	gen_shift_out_0_2/tmp_reg[2]/C	p[3]	4.058	3.258	0.800	∞				0.000
Path 8	∞	2	2	1	gen_shift_out_3_1/tmp_reg[1]/C	p[4]	4.058	3.258	0.800	∞				0.000
Path 9	∞	2	3	2	b[0]	gen_shift_out_0_2/mp_reg[7].sr[9]/D	2.239	1.106	1.133	∞	input port clock			0.000
Path 10	∞	2	2	2	gen_rows[2].gen_a/tmp_reg[1]/C	gen_shift_out_3_2/mp_reg[0].sr[2]/D	1.862	0.777	1.085	∞				0.000



We observe that the critical path has a total propagation delay of 4.076 ns.

Bonus: Programming the Zybo Board and Experimentation

To successfully complete this initial introduction to the Zybo board, we followed the provided instructions and verified the correct operation of the system by observing an LED blinking.

We then modified the provided code in order to gain a better understanding of its operation. More specifically:

(The code is included at this point to make the report easier to follow, since variables from the code are referenced in the subsequent discussion.)

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use ieee.std_logic_unsigned.all;

entity blink_led is
    generic (
        COUNTER_LIMIT : natural := 28
    );
    port (
        -----
        -- CONTROL SIGNALS
        i_clk  : in    std_logic;
        -----
        -- MAIN SIGNALS
        o_led0 : out   std_logic
    );
end entity;

architecture behavioral_arch of blink_led is

    signal count_reg : std_logic_vector(COUNTER_LIMIT-1 downto 0) := (others => '0');

begin

    BLINK_LED_PROCESS : process(i_clk)
    begin
        if (rising_edge(i_clk)) then
            count_reg <= count_reg + 1;
        end if;
    end process;

    o_led0 <= count_reg(COUNTER_LIMIT-1);

end behavioral_arch;
```

We observed that the value of the COUNTER_LIMIT variable directly affects the frequency at which the LED blinks. Specifically, the LED is on when the MSB of count_reg is 1 and off when it is 0, while the width of count_reg is determined by the value of COUNTER_LIMIT.

Consequently, as we increased the value of COUNTER_LIMIT, the LED blinked more slowly, whereas decreasing it caused the LED to blink faster. In fact, below a certain value, the switching became so fast that the LED appeared to the human eye to remain continuously on.

Furthermore, by slightly modifying the original code, we were able to achieve a duty cycle different from 50%. Instead of keeping the LED on and off for equal amounts of time, we were able to define different durations for these two states by comparing the counter value with an appropriate threshold, as shown below:

```
o_led0 <= '1' when count_reg < threshold else '0';
```

In this way, the LED remains on only for a selected portion of the counter's overall operating period, allowing the implementation of a duty cycle different from 50%.

The complete modified code is presented below:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity blink_led is
    generic (
        COUNTER_LIMIT : natural := 28
    );
    port (
        -----
        -- CONTROL SIGNALS
        i_clk    : in    std_logic;
        -----
        -- MAIN SIGNALS
        o_led0   : out   std_logic
    );
end entity;

architecture behavioral_arch of blink_led is

    signal count_reg : unsigned(COUNTER_LIMIT-1 downto 0) := (others => '0');
    signal threshold : unsigned(COUNTER_LIMIT-1 downto 0) := to_unsigned(192, COUNTER_LIMIT);

begin

    BLINK_LED_PROCESS : process(i_clk)
    begin
        if (rising_edge(i_clk)) then
            count_reg <= count_reg + 1;
        end if;
    end process;

    o_led0 <= '1' when count_reg < threshold else '0';

end behavioral_arch;
```

Finally, an additional observation that arose afterwards, but unfortunately was not tested in practice, is that by modifying the constraints file, it would be possible to drive additional LEDs so that they blink as well. Specifically, by uncommenting the corresponding lines and assigning the appropriate port names to the LEDs that are already declared but commented out below the active LED, it would be possible, with the necessary modifications to the design source code as well, to control multiple LEDs.

```
# Clock signal
set_property -dict { PACKAGE_PIN L16    IOSTANDARD LVCMOS33 } [get_ports { i_clk }];
create_clock -add -name sys_clk_pin -period 8.00 -waveform {0 4} [get_ports { i_clk }];

# LEDs
set_property -dict { PACKAGE_PIN M14    IOSTANDARD LVCMOS33 } [get_ports { o_led0 }];
#set_property -dict { PACKAGE_PIN M15    IOSTANDARD LVCMOS33 } [get_ports { led[1] }];
```

```
#set_property -dict { PACKAGE_PIN G14    IOSTANDARD LVCMOS33 } [get_ports { led[2] }];  
#set_property -dict { PACKAGE_PIN D18    IOSTANDARD LVCMOS33 } [get_ports { led[3] }];
```